

VEHICLE ANTI-THEFT SYSTEM

Project Report

Microprocessors | Project Topic #25

Kwame Nkrumah University of Science and Technology

March 2026

Group 8

Member	Index Number
Kwame Adom Asuma-Perprem	7266023
Francis Arthur	7264823
Hammond Jude Emmanuel Nii Armah	7272423
Bruce Lamptey David Aseda	7268623
Hardi Ruhaina	7272523
Asante-Opoku Francis	7265323
Dafeamekpor Michel Selorm	7269023
Kejanga Ajigeweh	7273423
Arloo Priscilla Afiba	7264523

1. Introduction

This report documents the design, implementation, and results of a Vehicle Anti-Theft System developed by Group 8 for the Microprocessors course, Project Topic #25. The objective was to build a system capable of monitoring a vehicle's geographic location in real time, detecting when it leaves a predefined safe area, and immediately alerting the owner.

The system was implemented as a simulation in Tinkercad using an Arduino Uno R3 microcontroller. Since GPS satellite receivers and GSM cellular modules cannot operate in any software simulation environment, both hardware components were replaced with functionally equivalent substitutes that preserve the system's logic, data format, and output behaviour in full. This approach is discussed in detail in Section 3.

2. Problem Statement

Vehicle theft remains one of the most frequently reported property crimes in urban environments. Conventional security measures such as steering locks, immobilisers, and passive alarms act as deterrents but provide no real-time location information once a vehicle has been moved. An owner has no way to determine where their vehicle is or when it left a known safe area.

A GPS-based anti-theft system addresses this directly. By continuously reading the vehicle's geographic coordinates and comparing them against a predefined geofence boundary, the system can detect the exact moment the vehicle exits the permitted zone and trigger an immediate alert. In a production deployment, this alert is transmitted as an SMS via a GSM module, delivering the breach coordinates directly to the owner's phone.

This project demonstrates a complete working simulation of that system: location monitoring, geofence evaluation, and multi-channel output, covering visual status display, owner notification, and audible alarm, all operating correctly in sequence.

3. Methodology

3.1 System Architecture

The system is structured across three layers: input, processing, and output. The input layer provides GPS coordinate data. The processing layer runs the geofence evaluation logic on each coordinate. The output layer drives the two LCD displays and the piezo buzzer based on the result of that evaluation.

The Arduino Uno R3 serves as the sole processing unit. It executes the main program loop, which reads one coordinate pair every two seconds, formats it for display, evaluates it against the safe zone boundaries, and updates all outputs accordingly.

3.2 GPS Simulation

A Neo-6M GPS module receives radio signals from satellites in low earth orbit and decodes them into NMEA sentences containing the current latitude and longitude. Since no simulation platform can generate satellite signals, the Neo-6M cannot be included in the circuit.

The substitute is a hardcoded array of 20 real GPS coordinates representing a vehicle route within and around KNUST campus in Kumasi. Each coordinate pair is read and processed identically to how the Arduino would handle live data from a GPS module, leaving the downstream logic unchanged. This technique is standard practice in embedded systems development and is referred to as sensor simulation.

The 20 coordinates begin at KNUST central campus and move progressively north-east. Coordinates 1 through 12 remain inside the defined safe zone; coordinate 13 is the first to cross the boundary, and coordinates 14 through 20 continue outside the zone.

3.3 GSM Simulation

In a production system, a SIM800L GSM module sends an SMS to the vehicle owner the moment the geofence is breached, containing the alert message and the exact breach coordinates. GSM requires a live SIM card and a real mobile network and cannot be simulated.

The substitute is a second 16x2 I2C LCD display. On breach, LCD 2 first shows an owner alert message, then switches to display the breach coordinates, fulfilling the same informational purpose as an SMS notification within the constraints of the simulator. The conclusion explicitly acknowledges that a real deployment replaces this LCD with a SIM800L GSM module.

3.4 Geofence Definition and Approach

The safe zone is a rectangular bounding box defined by four coordinate constants. Any incoming coordinate that violates at least one boundary condition is classified as outside the zone and triggers the alert state.

Constant	Value	Boundary
LAT_MIN	6.6720	Southern limit: below this latitude is outside the zone
LAT_MAX	6.6775	Northern limit: above this latitude is outside the zone
LON_MIN	-1.5720	Western limit: below this longitude is outside the zone
LON_MAX	-1.5630	Eastern limit: above this longitude is outside the zone

The rectangular bounding box was chosen over the circular geofence alternative for three reasons. First, KNUST campus has an approximately rectangular geographic footprint, making this approach a natural fit. Second, the boundary check requires only four simple comparisons with no trigonometric computation, which is appropriate for the ATmega328P processor. Third, the approach produces deterministic SAFE/ALERT transitions at predictable coordinate steps, making the simulation straightforward to verify.

Known limitation: The alternative approach, a circular geofence using the Haversine formula, computes the actual great-circle distance between the vehicle and a defined centre point, producing a boundary that is geometrically accurate at any latitude. A rectangular bounding box becomes physically distorted at higher latitudes because one degree of longitude represents progressively shorter distances than one degree of latitude as you move away from the equator. At KNUST's latitude of approximately 6.67 degrees North, this distortion is less than 0.5% and has no practical effect at campus scale. For a production deployment at higher latitudes or covering a substantially larger area, the Haversine-based circular geofence would be the more accurate choice.

3.5 System Logic Flow

The Arduino executes the following sequence on each pass through the main loop:

- Read the next latitude and longitude from the coordinate arrays
- Display the coordinate pair on LCD 1, with latitude on row 0 and longitude on row 1
- Pause 1.2 seconds to allow the coordinates to be read clearly
- Evaluate the coordinate against all four geofence boundaries
- If inside the safe zone: LCD 1 shows SAFE status, LCD 2 shows standby message, piezo stays silent

- If outside the safe zone: LCD 1 shows ALERT, piezo siren activates, LCD 2 shows owner alert then breach coordinates
- Advance the coordinate index and repeat
- After coordinate 20, hold the final ALERT state permanently

4. Components

All components used in this project are available natively in Tinkercad. No external libraries or downloads are required beyond the LiquidCrystal I2C library available in Tinkercad's built-in library panel.

Component	Qty	Role	Notes
Arduino Uno R3	1	Central microcontroller	ATmega328P, AVR architecture
16x2 LCD (I2C)	2	LCD 1: coordinates and status; LCD 2: owner alert	PCF8574-based. Addresses 0x27 and 0x26
Piezo Buzzer	1	Audible siren alarm	Driven via tone()/noTone()
Breadboard (830-pt)	1	Power, ground, and I2C routing	SDA/SCL via shared breadboard rows
220 Ohm Resistor	1	Current limiting for piezo	In series with piezo positive leg
Jumper Wires	~20	All connections	Red = 5V, Black = GND

Note on absent components: No GPS or GSM module appears in the circuit. The Neo-6M GPS module requires live satellite signals and cannot be simulated. The SIM800L GSM module requires a live SIM card and mobile network and cannot be simulated. Both are substituted as described in Section 3.

5. Code Overview

5.1 Library and LCD Initialisation

The LiquidCrystal_I2C library handles I2C communication with both displays. Both share the SDA line on Arduino pin A4 and the SCL line on pin A5, and are distinguished by their I2C addresses.

```
#include <LiquidCrystal_I2C.h>
LiquidCrystal_I2C lcd1(0x27, 16, 2); // Coordinates and zone status
LiquidCrystal_I2C lcd2(0x26, 16, 2); // Owner alert and breach
coordinates
```

5.2 Geofence Constants and Coordinate Arrays

The safe zone is defined by four preprocessor constants. Two parallel float arrays store the 20-step coordinate route, covering 12 steps inside the boundary and 8 outside.

```
#define LAT_MIN 6.6720 // Southern boundary
#define LAT_MAX 6.6775 // Northern boundary
#define LON_MIN -1.5720 // Western boundary
#define LON_MAX -1.5630 // Eastern boundary
const float latCoords[20] = { 6.6730, 6.6735, ... 6.6868 };
const float lonCoords[20] = { -1.5700, -1.5693, ... -1.5472 };
```

5.3 Geofence Evaluation

The `isInsideSafeZone()` function performs four boundary comparisons and returns true only if all four pass simultaneously. Failure of any single condition classifies the coordinate as outside the zone.

```
bool isInsideSafeZone(float lat, float lon) {  
    return (lat >= LAT_MIN && lat <= LAT_MAX &&  
            lon >= LON_MIN && lon <= LON_MAX);  
}
```

5.4 Coordinate Formatting

The `fmtCoord()` function converts a float coordinate to a 4-decimal-place string for LCD display. It uses `round()` to prevent floating-point accumulation errors that would cause inconsistent display values on certain coordinates.

5.5 Siren Function

The `soundSiren()` function produces a rising-and-falling tone by sweeping the piezo frequency from 500 Hz to 2000 Hz and back in 50 Hz increments with a 15 ms delay per step. This generates an audible wail in the Tinkercad simulation and represents the alarm condition for the vehicle owner.

```
void soundSiren() {  
    for (int f = 500; f <= 2000; f += 50) { tone(BUZZER_PIN, f);  
        delay(15); }  
    for (int f = 2000; f >= 500; f -= 50) { tone(BUZZER_PIN, f);  
        delay(15); }  
}
```

5.6 Setup and Main Loop

The `setup()` function initialises both LCDs with `init()` and `backlight()`, displays a startup splash for 2 seconds, then clears both screens to begin the main loop. The `loop()` function steps through the coordinate array sequentially, calling the geofence check on each pair and driving all three outputs based on the result. Once the 20th coordinate has been processed, the loop holds the final state permanently by returning early on every subsequent call.

6. Results

The simulation executed correctly through all 20 coordinate steps. The observed behaviour at each stage was as follows.

On startup, both LCDs illuminated simultaneously. LCD 1 displayed 'Anti-Theft Sys / Initialising' and LCD 2 displayed 'Group 8 COE / GPS Tracking ON' for the 2-second splash duration, confirming correct I2C initialisation on both addresses.

During coordinates 1 through 12, LCD 1 updated with each new coordinate pair and then displayed 'Status: SAFE / Inside Zone'. LCD 2 showed 'Zone: ACTIVE / Vehicle OK' throughout. The piezo remained silent. Each coordinate was evaluated correctly as inside the bounding box.

At coordinate 13 (6.6778 degrees N, -1.5626 degrees E), the northern and eastern boundaries were both exceeded. LCD 1 switched immediately to '!! ALERT !! / Outside Zone'. The `soundSiren()` function activated, producing the rising-and-falling tone through the piezo. LCD 2 displayed 'OWNER ALERT! / Veh left zone', then switched after one second to show the exact breach coordinates.

Coordinates 14 through 20 all triggered the ALERT state as expected, with the siren activating on each step. After coordinate 20, the system held the final ALERT state with the siren continuing, confirming the permanent-hold behaviour of the termination condition.

The simulation confirmed correct operation across all logic paths: coordinate display, geofence evaluation, safe-state outputs, alert-state outputs, and terminal-state handling.

7. Conclusion

This project successfully demonstrated the core functionality of a GPS-based vehicle anti-theft system on an Arduino Uno R3 platform. The simulation correctly implemented all three layers of the system: GPS coordinate input via sensor simulation, rectangular geofence evaluation in the processing layer, and tri-channel output via LCD status display, LCD owner notification, and piezo siren alarm.

The system's logic is identical to that of a production anti-theft system. The only differences are the data source, which uses a coordinate array in place of a Neo-6M GPS module, and the notification channel, which uses an LCD in place of a SIM800L GSM module. Neither substitution changes the geofence logic, the output sequencing, or the system's ability to demonstrate the intended concept.

The rectangular bounding box method used in this project is accurate and appropriate at KNUST's latitude of 6.67 degrees North, where longitudinal distortion is below 0.5%. For production deployments at higher latitudes or over larger areas, upgrading to a Haversine-based circular geofence would improve geometric accuracy and is identified as the primary technical improvement for future development.

Further improvements would include:

- Integrating a relay module to cut the vehicle's ignition circuit on alert
- Replacing the coordinate array with a live Neo-6M GPS module
- Replacing LCD 2 with a SIM800L GSM module for SMS notification
- Adding EEPROM logging to retain breach records after power loss
- Implementing two-way GSM communication for remote arm/disarm capability